diagram): the ASK chairperson, the Airlines smart contract (Airlines.sol), the two airlines (`fromAirline` and `toAirline`) involved in the seat change (in this particular scenario), and the off-chain display. The airlines go about their regular business. They send only the information about the seat change to the smart contract. Only transactions related to the seat change are recorded on the blockchain, via the invocation of smart contract functions. All the Txs recorded on the blockchain are shown within the ellipse in figure 6.13. Also, observe the off-chain operations indicated by dotted arrows. These operations include the request and response processing with the airlines' own databases and seat availability notifications to the ASK display board. Take a few minutes to review the sequence of exchanges in figure 6.13 before continuing with your exploration.

> **DESIGN PRINCIPLE 9**: Use a UML sequence diagram to represent the sequence(s) in which functions within a smart contract may (and can) be called. The sequence diagram captures the dynamic operations of a system.

6.4.2 Airlines smart contract

The updated Airlines smart contract (Airlines.sol) is shown in listing 6.2. You'll find a copy of this listing in the codebase for this chapter; you can copy it into the Remix IDE to do a bit of exploration. The code illustrates how you can define state variables as an on-chain record representing parameters of operations happening off-chain. You do this by defining a struct data type for recording the state changes effected by the `ASKRequest()` and `ASKResponse()` functions. Observe that the fields in the `structs` `reqStruc` and `respStruc` match one-to-one the parameters of the functions `ASKRequest()` and `ASKResponse()`, respectively. These data structures facilitate the recording of information about a request and a response and